

# Cast Expressions

A cast expression can be used to explicitly convert from one variable type to another, eg

```
double x = 3.14; char ch = 'a'; int y, z;  
y = (int)x; // convert x to an int and set y to the result  
z = (int)ch; // convert ch to an int and set z to the result
```

Here the cast (int) instructs the compiler to convert the variable to an int -- the result is the integer "equivalent" of the variable

Note that the variable types of x and ch don't change. It's just the result of the cast expression that has the type of int.

Conversion between most variable types is possible, but might not make sense in some applications (it will be consistent though)

It's also possible some information might be lost if converting from a type that has more bytes, eg

```
char ch; // char is 1 byte with a range of -128 to 127  
int x = 3000; // int is 4 bytes with a range of +/- 2 billion  
ch = (char)x; // x is out of range of ch (only 1st byte of x is used)
```

--> loss of some information (answer doesn't make sense)

Note that character constants convert to specific integers associated with the ASCII standard, see the table below  
<https://en.cppreference.com/w/cpp/language/ascii>

This can be useful for converting text to int, etc.

For some conversions, where the variable types are similar, an explicit cast statement isn't required

In this case an automatic / "implicit" conversion is performed

A compiler warning might be generated, which can be removed using a cast (or adjusting the compiler settings)

```
double d1, d2 = 3.40000000000001;  
float f1, f2 = 3.6;  
int i1, i2 = 5;
```

```
d1 = f2; // digits are added to d1 for extra digits of precision
```

```
f1 = d2; // extra digits of precision for d2 are truncated
```

```
i1 = f2; // f2 is rounded down
```

```
i1 = d2; // d2 is rounded down
```

```
d1 = i2; // zeros are added to d1 for extra digits of precision
```

# Type Modifiers

A variable type modifier allows you to adjust a variable type, eg

```
short int x; // a 2 byte integer (range from -32768 to 32767)
```

Here "short" is a type modifier that reduces the size of "int"

The following type modifiers will be discussed in this lecture:  
short, long, unsigned, const -- the typical ranges are given in

[https://www.tutorialspoint.com/cplusplus/cpp\\_data\\_types.htm](https://www.tutorialspoint.com/cplusplus/cpp_data_types.htm)

Other type modifiers will be discussed later in the term

short: makes the data type smaller (less bytes, smaller range)  
if possible -- useful to reduce memory consumption, eg

```
short int x; // 2 byte integer on 32/64 bit CPU
```

long makes the data type larger (more bytes, larger range)  
if possible -- useful to increase range of variable type

```
long int y; // doubles the byte size of int on some CPU types
```

a long / short type modifier will change the size of a variable if  
possible -- in some cases the compiler will ignore the request, eg

```
sizeof(long int) == sizeof(int) for a 32/64 bit compiler
```

unsigned: restricts the range of a variable to positive or zero numbers. this also typically doubles the positive range, eg

unsigned int z; // range from 0 to 4294967295 on 32/64 bit CPU  
(size in bytes remains unchanged with unsigned)

unsigned char w = 237; // range, 0 to 255 (char is -128 to 127)



the main application of unsigned is to double the range and avoid negative numbers in applications where they are not valid

const: fixes a variable to be a constant (ie can't be changed from its declared initialization value), eg

const double PI = 3.14159;

PI = 1.7; // this can't be done -- generates a compiler error

More than one modifier can be applied to a variable type, eg.

```
const unsigned int q = 12345; // constant positive int
```

```
unsigned long int p = 1999; // unsigned large int
```

```
const unsigned short int s = 270; // constant unsigned short int
```









